Isolating the Effects of Deep Q-Network Extensions: A Paired Randomised Controlled Trial on MinAtar

Matimu Khosa
School of Computer Science and Applied Mathematics
University of the Witwatersrand
Johannesburg, South Africa
2716924@students.wits.ac.za
Supervised by Steven James and Benjamin Rosman

***Abstract*—Extensions to the deep Q-network (DQN) are usually evaluated with their own tuned hyperparameters and a handful of random seeds, so a reported gain may reflect the configuration rather than the extension. We evaluate four extensions, namely Double DQN, prioritised experience replay, multi-step returns and Averaged DQN, in a paired randomised controlled trial. Thirty hyperparameter configurations were sampled from a shared population, each with its own seed, and every configuration was trained with DQN and with each extension on four MinAtar games, giving 600 runs of five million frames. The primary outcome was the area under the greedy evaluation curve, compared with DQN by paired t-tests with a Bonferroni correction across all sixteen comparisons. Averaged DQN improved on DQN in Asterix, Freeway and Space Invaders, by 58, 21 and 19 per cent. Double DQN improved only in Space Invaders, by 11 per cent, and the Wilcoxon signed-rank test did not confirm this. Prioritised replay was worse in Freeway and Space Invaders, and multi-step returns were worse in Asterix, Freeway and Space Invaders, by up to 80 per cent. No extension differed detectably from DQN in Breakout. The sampled learning rate was associated with larger differences in performance than fifteen of the sixteen treatment effects, and evaluating only the final checkpoint confirmed three of the nine findings. Under a shared training recipe, Averaged DQN was the only reliable improvement.**

***Index Terms*—deep reinforcement learning, deep Q-network, experimental design, randomised controlled trial, MinAtar, sample efficiency**

I. Introduction

The deep Q-network (DQN) of Mnih et al. [2015] learnt to play Atari 2600 games from pixels, using two stabilising devices: experience replay and a periodically updated target network. Many extensions followed, each changing one part of the algorithm. Double DQN changes how the bootstrap target is computed [van Hasselt et al. 2016], prioritised experience replay (PER) changes which transitions are replayed [Schaul et al. 2016], multi-step (n-step) returns lengthen the horizon of the target [Hernandez-Garcia and Sutton 2019; Sutton and Barto 2018], and Averaged DQN averages the estimates of several earlier target networks [Anschel et al. 2017]. Rainbow combined several such components and, in an ablation, found prioritised replay and multi-step learning to be the two most important [Hessel et al. 2018].

Each extension was, however, introduced with its own training recipe and evaluated on its own configuration. Deep reinforcement learning results are sensitive to hyperparameters, random seeds and implementation details [Henderson et al. 2018], evaluation protocols differ between papers [Machado et al. 2018], and comparisons often rest on few runs and point estimates [Colas et al. 2018; Agarwal et al. 2021]. When an extension outperforms DQN under one tuned configuration, it is therefore hard to say whether the gain belongs to the extension or to the configuration.

We address this with a randomised controlled trial in the sense of Pretorius et al. [2020]. The individuals of the trial are hyperparameter configurations sampled from a shared population, and every individual receives every treatment. Each comparison with DQN is then paired on the same configuration and seed, which removes the variation between configurations from the comparison while still averaging over the population. We use the MinAtar suite [Young and Tian 2019], whose small observations make 600 full training runs feasible on CPUs.

We ask two research questions. *RQ1:* across a population of configurations and under one shared training recipe, does each extension change sample efficiency, measured as the area under the evaluation curve (AUC), relative to DQN? *RQ2:* does a curve-based, population-level evaluation reach different conclusions from an evaluation based on a single checkpoint? Our contributions are:

- a paired trial of four DQN extensions on 30 sampled configurations and four MinAtar games (600 runs, 3×10^9 frames), with code, raw results and provenance released[1];
- evidence that Averaged DQN is the only extension that improves on DQN consistently, that the benefit of Double DQN is small, and that PER and n-step returns are harmful under the shared recipe;
- evidence that effects vary across the population: the learning rate is associated with larger differences than

[1]https://github.com/2716924/research_project

almost all treatment effects, and the n-step deficit is concentrated at the lowest learning rate;
- a demonstration that single-checkpoint endpoints confirm only some of the AUC findings.

Section II reviews background and related work, Section III describes the trial, Section IV reports the results, Section V discusses them and Section VI states the limitations.

II. Background and Related Work

A. Deep Q-Networks

An agent interacting with a Markov decision process observes a state s_t, takes an action a_t, receives a reward r_{t+1} and moves to s_{t+1}; it seeks to maximise the expected return discounted by $\gamma \in [0, 1)$ [Sutton and Barto 2018]. Q-learning [Watkins and Dayan 1992] estimates the optimal action-value function by bootstrapping from $\max_{a'} Q(s', a')$. DQN [Mnih et al. 2015] approximates Q with a neural network with parameters θ, stores transitions in a replay buffer $\mathcal{D}$ [Lin 1992] and minimises

$$L(\theta) = \mathbb{E}_{(s,a,r,s') \sim U(\mathcal{D})} \big[(y - Q(s, a; \theta))^2 \big] \tag{1}$$

with the target

$$y = r + \gamma \max_{a'} Q(s', a'; \theta^-), \tag{2}$$

where θ^- are the parameters of a target network copied from θ every C steps, and $y = r$ when s' is terminal.

B. The Four Extensions

Double DQN [van Hasselt et al. 2016]. The maximum in (2) uses the same estimates to select and to evaluate the next action, which biases the target upwards. Double DQN selects the action with the online network and evaluates it with the target network:

$$y = r + \gamma \, Q\big(s', \arg\max_{a'} Q(s', a'; \theta); \theta^-\big). \tag{3}$$

Prioritised experience replay [Schaul et al. 2016]. Instead of sampling uniformly, PER samples transition i with probability

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha}, \qquad p_i = |\delta_i| + \epsilon, \tag{4}$$

where δ_i is the latest temporal-difference (TD) error of the transition. New transitions receive the largest priority seen so far. Importance-sampling weights $w_i = (|\mathcal{D}| \, P(i))^{-\beta} / \max_j w_j$ correct the bias of non-uniform sampling, with β annealed linearly to 1. For the proportional variant used here, Schaul et al. [2016] chose $\alpha = 0.6$ and $\beta_0 = 0.4$ by a coarse grid search, and they reduced the step size by a factor of four relative to their baseline because prioritised replay produces larger gradients.

Multi-step returns [Hernandez-Garcia and Sutton 2019; Sutton and Barto 2018]. The n-step target sums n observed rewards before bootstrapping:

$$y = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n \max_{a'} Q(s_{t+n}, a'; \theta^-). \tag{5}$$

Without an off-policy correction, as in Rainbow [Hessel et al. 2018], the reward sum follows the exploratory behaviour policy rather than the greedy policy, which trades bias for faster propagation of reward.

Averaged DQN [Anschel et al. 2017]. Averaged DQN keeps the last K target networks $\theta_1^-, \dots, \theta_K^-$ and averages their action values before taking the maximum:

$$y = r + \gamma \max_{a'} \frac{1}{K} \sum_{k=1}^{K} Q(s', a'; \theta_k^-). \tag{6}$$

This reduces the variance of the target approximation error at the cost of K forward passes per target, with the same number of gradient updates as DQN [Anschel et al. 2017].

C. Component Studies

Rainbow's ablation on 57 Atari games found that removing prioritised replay or multi-step learning caused the largest drops in median performance [Hessel et al. 2018]. Obando-Ceron and Castro [2021] repeated the additive and ablative studies on smaller environments, including the five MinAtar games, and found multi-step returns to be the component that most improved DQN on MinAtar. Their agents used the Adam optimiser [Kingma and Ba 2015], fixed hyperparameters and 10 runs per setting; they also observed on Atari that, with RMSProp, the Huber loss tends to perform better than the mean squared error (MSE). Fedus et al. [2020] found that uncorrected n-step returns benefit uniquely from a larger replay capacity, whereas DQN does not. Fujimoto et al. [2020] showed that a loss evaluated on non-uniformly sampled data can be rewritten as a uniformly sampled loss with the same expected gradient, which ties the effect of PER to the loss it is combined with.

D. Evaluation Methodology

Henderson et al. [2018] showed that hyperparameter settings, random seeds, environment properties and codebases can each change reported performance substantially. Machado et al. [2018] revisited the evaluation protocols of the Arcade Learning Environment and introduced sticky actions to make it stochastic. Colas et al. [2018] related the number of random seeds to the error rates of statistical tests, and Agarwal et al. [2021] showed that conclusions drawn from point estimates over few runs can differ substantially from those of a more thorough statistical analysis, and advocated interval estimates. Pretorius et al. [2020] brought the randomised controlled trial to deep learning: they sampled network designs from a population, trained identical copies of each design under every intervention and tested the differences, over more than 12,000 trained networks. We apply this design to DQN extensions and exploit the pairing explicitly.

III. Method

A. Trial Design

Fig. 1 summarises the design. The population Λ is the Cartesian product of the candidate values of six shared hyperparameters (Table I). Each of the $N = 30$ individuals λ_j was

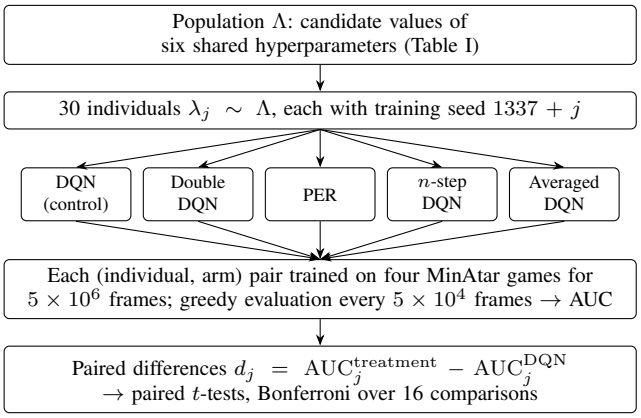


Fig. 1. Design of the trial. Every individual (a hyperparameter configuration with its own seed) is trained under all five arms, so each treatment is compared with DQN on the same configuration.

TABLE I
SHARED HYPERPARAMETER POPULATION AND THE SAMPLED INDIVIDUALS

Hyperparameter	Values (individuals in sample)
Learning rate	10^{-4} (7), $5{\times}10^{-4}$ (13), 10^{-3} (10)
Replay buffer capacity	10^4 (15), $5{\times}10^4$ (5), 10^5 (10)
Mini-batch size	32 (15), 64 (15)
Target update period (steps)	500 (8), 1000 (11), 2000 (11)
ε-decay duration (steps)	$5{\times}10^4$ (11), 10^5 (7), $2{\times}10^5$ (12)
Hidden layers	1 (16), 2 (14)

drawn by sampling every hyperparameter independently and uniformly from its candidates with a seeded generator, and was given its own training seed $1337 + j$. This is a simple random sample, and Table I shows the resulting counts, which are unbalanced for the replay buffer capacity. Every run filled the buffer with 5,000 transitions, collected by uniformly random actions, before learning began.

Five arms were trained on every individual: DQN, the control, and four treatments, each of which changes one component (Section II-B). The treatment-specific settings were fixed at published values rather than sampled: $\alpha = 0.6$, $\beta_0 = 0.4$ and $\epsilon = 10^{-6}$ for PER [Schaul et al. 2016], $n = 3$ for n-step DQN [Hessel et al. 2018] and $K = 10$ for Averaged DQN [Anschel et al. 2017]. Within an individual, all arms share the configuration, the training seed and the evaluation seeds, so every comparison with DQN is paired.

The arms were trained on four MinAtar games [Young and Tian 2019]: Breakout, Asterix, Freeway and Space Invaders (version 1, with the minimal action set). MinAtar's default settings were kept, including sticky actions with probability 0.1 [Young and Tian 2019] and difficulty ramping in the games that support it. Seaquest was also run but is excluded: in every arm in which it was run, a greedy evaluation episode exceeded the 100,000-step limit, because MinAtar never ends a Seaquest episode in which the submarine stays on the surface, where it starts. The trial therefore comprises $4 \times 5 \times 30 = 600$ runs.

TABLE II
TRAINING SETTINGS

Setting	Value
Network	convolution (16 filters, 3×3, stride 1), then 1 or 2 hidden layers of 128 units (ReLU)
Optimiser, loss	RMSProp (decay 0.99, $\epsilon = 10^{-8}$, no momentum); MSE
Discount, exploration	$\gamma = 0.99$; ε from 1.0 to 0.1, linear over the sampled duration
Updates	one per step after 5,000 random transitions
Training length	5×10^6 frames (no frame skipping)
Evaluation	every 5×10^4 frames; 10 greedy episodes with seeds 1000–1009
PER	proportional; $\alpha = 0.6$; β from 0.4 to 1 over training; $\epsilon = 10^{-6}$
n-step DQN	$n = 3$; no off-policy correction
Averaged DQN	$K = 10$ target networks, one stored at each target update

B. Agent and Shared Training Recipe

Table II lists the training settings. All arms use the network of the MinAtar DQN baseline [Young and Tian 2019]: one convolutional layer with 16 3×3 filters and stride 1, followed by fully connected hidden layers of 128 units, with ReLU activations; whether there are one or two hidden layers is a sampled hyperparameter. Young and Tian [2019] chose 16 filters and 128 units as a quarter of the corresponding layers of DQN [Mnih et al. 2015]. Training uses RMSProp [Tieleman and Hinton 2012] with the MSE loss (1) and one gradient update per environment step once learning has begun. Exploration is ε-greedy, with ε annealed linearly from 1.0 to 0.1 over the sampled duration. MinAtar has no frame skipping, so a frame is one environment step, and each run lasts 5×10^6 frames, as in the MinAtar baselines [Young and Tian 2019].

The recipe is deliberately shared and is not the recipe of any one paper: DQN is usually trained with the Huber loss [Obando-Ceron and Castro 2021], and Schaul et al. [2016] lowered the step size for PER. Changing one component under a fixed recipe is what isolates its effect, but it also means that the results concern these components under this recipe.

The treatments were implemented as follows. PER stores priorities in a sum tree, draws each mini-batch by stratified sampling over equal segments of the total priority, normalises the importance weights by the largest possible weight, anneals β linearly over the 5×10^6 frames and averages the weighted squared TD errors. n-step DQN stores, for each transition, the discounted sum of the next three rewards and the state three steps later; at the end of an episode, the remaining shorter returns are stored without a bootstrap. Averaged DQN stores a copy of the target network at every target update and averages the last ten copies as in (6), or all of them while fewer than ten exist.

C. Evaluation and Primary Outcome

Every 5×10^4 frames, the online network was evaluated greedily ($\varepsilon = 0$) for 10 episodes, with the same environment seeds (1000–1009) for every individual, arm and checkpoint.

The mean return G_i at checkpoint i is one point on the evaluation curve, giving 100 points per run. An evaluation episode longer than 10^5 steps aborts the run without recording a score. The primary outcome is the trapezoidal area under the curve,

$$\mathrm{AUC} = \sum_{i=1}^{99} \frac{G_i + G_{i+1}}{2}\,\Delta x, \qquad \Delta x = 5 \times 10^4 \text{ frames}, \quad (7)$$

reported in units of 10^6 (return $\times$ frames). The AUC rewards both fast learning and a high final level: divided by the 4.95×10^6 frames it spans, it is the average evaluation return over training.

D. Statistical Analysis

The statistical unit is the individual; the ten evaluation episodes and the 100 checkpoints are repeated measurements within a run, not replicates. For each game and treatment, we form the 30 paired differences $d_j = \mathrm{AUC}_j^{\text{treatment}} - \mathrm{AUC}_j^{\text{DQN}}$ and test whether their mean is zero with a two-sided paired t-test. We treat all $4 \times 4 = 16$ comparisons as one family and apply the Bonferroni correction [Dunn 1961], $\alpha = 0.05/16 = 0.003125$, with matching simultaneous 99.69% t-intervals. We report each effect as the mean difference as a percentage of the DQN mean, with the standardised effect $d_z = \bar{d}/s_d$ [Lakens 2013] and the number of individuals for which the treatment beat DQN.

This analysis departs from our research proposal in two ways. First, the proposal applied the Bonferroni correction within each game ($\alpha = 0.0125$); because our conclusions are drawn across games, we use the stricter family-wide threshold. Second, the proposal would have replaced the t-test by the Wilcoxon signed-rank test whenever a Shapiro–Wilk test [Shapiro and Wilk 1965] rejected normality of the differences. Choosing the test after a preliminary normality test alters the conditional error rates of the test that follows [Rochon et al. 2012], so we instead report both tests for every comparison, with the t-test as primary and the Wilcoxon test [Wilcoxon 1945] as a sensitivity analysis. For reference, Shapiro–Wilk rejected normality at the 0.05 level for 6 of the 16 comparisons.

Two secondary endpoints, analysed in the same way and treated as exploratory, address RQ2: the mean of the last ten evaluations (the final 5×10^5 frames) and the final evaluation alone. The bands in the learning curves are pointwise 95% percentile bootstrap intervals over individuals, from 2,000 resamples [Efron 1979].

E. Implementation and Data Provenance

The agents were implemented in PyTorch [Paszke et al. 2019] (version 2.13) with the Gymnasium [Towers et al. 2024] interface to MinAtar (version 1.0.15) and trained on CPUs, with four threads per job, on the MSL cluster at the University of the Witwatersrand. The code includes unit tests and analytic checks of every arm's target and replay computations, which run before each cluster job.

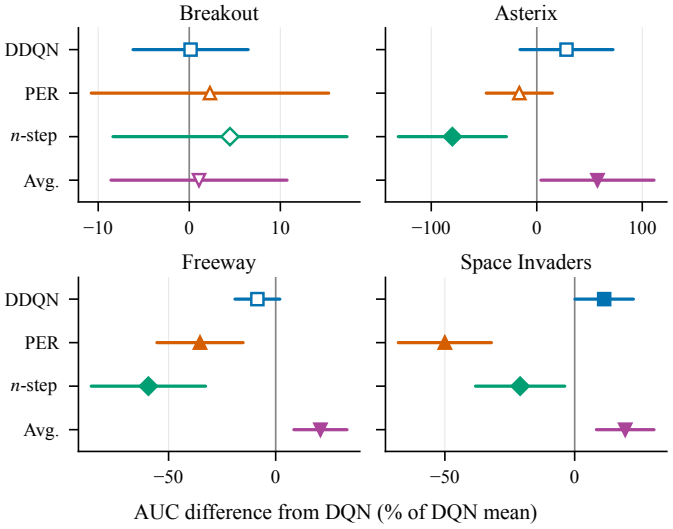


Fig. 2. Mean paired AUC difference from DQN, as a percentage of the DQN mean, with Bonferroni-adjusted 99.69% intervals. Filled markers: $p_t \leq 0.05/16$. The horizontal scales differ between games. DDQN: Double DQN; Avg.: Averaged DQN.

An initial run used a PER implementation that applied the priority exponent twice when inserting a new transition, giving it priority $p_{\max}^{\alpha^2}$ instead of $p_{\max}^{\alpha}$. After the fix, the PER, Double DQN, n-step and Averaged DQN arms were rerun with the corrected code. The three arms that do not use prioritised replay reproduced all 348 of their overlapping learning curves exactly, which shows that training is reproducible and that the fix affected only PER. The DQN arm, which was not rerun, is therefore taken from the initial run, as are n-step individuals 18–29 in Freeway, whose rerun job failed to start. Every stored AUC was recomputed from its curve, every individual was checked to have the same configuration and seed in every arm, and all tests were reproduced by an independent script.

IV. Results

A. Primary Outcome

Table III and Fig. 2 report the 16 comparisons, nine of which are significant at the family-wide threshold. Averaged DQN had a higher AUC than DQN in Asterix (+57.5%), Freeway (+20.9%) and Space Invaders (+19.5%), and beat DQN on 25, 24 and 26 of the 30 individuals. Double DQN was better only in Space Invaders (+11.4%, 21 of 30 individuals); its differences in Asterix (+28.2%, $p_t = 0.048$) and Freeway (−8.5%, $p_t = 0.014$) were not significant. PER was worse in Freeway (−35.3%) and Space Invaders (−50.0%), beating DQN on only 3 and 1 individuals. n-step DQN was worse in Asterix (−80.0%), Freeway (−59.4%) and Space Invaders (−21.0%), with 2, 0 and 6 wins. The significant effects have $|d_z|$ between 0.59 and 1.64.

In Breakout no treatment differed significantly from DQN. The point estimates lie between +0.1% and +4.5%, but the intervals extend from −10.8% to +17.3%, so the data do not show that the extensions are equivalent to DQN there either.

TABLE III
MEAN AUC (10^6, STANDARD DEVIATION IN PARENTHESES) AND PAIRED COMPARISONS WITH DQN ($n = 30$ PER ARM)

Game	Arm	AUC	Δ (%)	99.69% CI (%)	d_z	p_t	p_W	Better
Breakout	DQN	65.2 (16.6)						
	Double DQN	65.3 (16.9)	$+0.1$	$[-6.2, +6.5]$	$+0.01$	0.94	0.92	14/30
	PER	66.7 (18.3)	$+2.3$	$[-10.8, +15.3]$	$+0.10$	0.58	0.36	17/30
	n-step DQN	68.1 (20.0)	$+4.5$	$[-8.4, +17.3]$	$+0.20$	0.27	0.24	19/30
	Averaged DQN	65.9 (17.6)	$+1.1$	$[-8.6, +10.7]$	$+0.07$	0.72	0.72	17/30
Asterix	DQN	16.8 (14.1)						
	Double DQN	21.5 (21.7)	$+28.2$	$[-15.9, +72.3]$	$+0.38$	0.048	0.13	17/30
	PER	14.0 (9.6)	-16.5	$[-47.9, +14.8]$	-0.31	0.10	0.27	14/30
	n-step DQN	3.3 (2.4)	-80.0	$[-131.3, -28.8]$	-0.92	$\mathbf{2.3 \times 10^{-5}}$	$\mathbf{1.2 \times 10^{-6}}$	2/30
	Averaged DQN	26.4 (25.1)	$+57.5$	$[+3.9, +111.1]$	$+0.63$	**0.0017**	$\mathbf{4.2 \times 10^{-4}}$	25/30
Freeway	DQN	159.8 (58.9)						
	Double DQN	146.2 (64.8)	-8.5	$[-19.0, +1.9]$	-0.48	0.014	0.019	10/30
	PER	103.4 (63.7)	-35.3	$[-55.5, -15.2]$	-1.03	$\mathbf{4.1 \times 10^{-6}}$	$\mathbf{1.0 \times 10^{-7}}$	3/30
	n-step DQN	64.9 (54.9)	-59.4	$[-86.1, -32.7]$	-1.31	$\mathbf{6.7 \times 10^{-8}}$	$\mathbf{1.9 \times 10^{-9}}$	0/30
	Averaged DQN	193.2 (52.4)	$+20.9$	$[+8.5, +33.3]$	$+0.99$	$\mathbf{7.7 \times 10^{-6}}$	$\mathbf{1.1 \times 10^{-5}}$	24/30
Space Invaders	DQN	168.8 (90.3)						
	Double DQN	188.0 (100.6)	$+11.4$	$[+0.1, +22.6]$	$+0.59$	**0.0029**	0.012	21/30
	PER	84.4 (65.4)	-50.0	$[-68.0, -32.1]$	-1.64	$\mathbf{7.1 \times 10^{-10}}$	$\mathbf{9.3 \times 10^{-9}}$	1/30
	n-step DQN	133.4 (106.0)	-21.0	$[-38.2, -3.8]$	-0.72	$\mathbf{4.7 \times 10^{-4}}$	$\mathbf{1.9 \times 10^{-4}}$	6/30
	Averaged DQN	201.7 (111.0)	$+19.5$	$[+8.4, +30.6]$	$+1.03$	$\mathbf{4.1 \times 10^{-6}}$	$\mathbf{3.8 \times 10^{-6}}$	26/30

Δ: mean paired difference as a percentage of the DQN mean. CI: Bonferroni-adjusted 99.69% t-interval on the same scale; it is symmetric, so it can extend below -100%. d_z: mean difference divided by the standard deviation of the differences. p_t, p_W: two-sided paired t-test and Wilcoxon signed-rank p-values; bold values are at most $0.05/16$. Better: individuals for which the treatment's AUC exceeds that of DQN.

B. Learning Dynamics

The learning curves (Fig. 3) show how these differences arise. In Breakout the curves rise quickly and then overlap for the rest of training, ending between 13.2 and 14.8 (mean of the last ten evaluations); n-step DQN learnt fastest early on, with a mean return of 14.0 against 12.0 for DQN at 5×10^5 frames. In Asterix, n-step DQN barely improved: its mean curve never exceeded 1.24, whereas DQN ended at 3.8. In Freeway, Averaged DQN led from early in training (31.6 against 20.1 for DQN at 5×10^5 frames) and ended highest (44.1 against 38.4). In Space Invaders, PER started level with DQN (23.6 against 23.5 at the first evaluation), fell to 13.0 by 10^6 frames and ended at about half the level of DQN (19.1 against 35.9).

C. Variation Across Configurations

Performance varied widely between configurations (Fig. 4). For DQN, the standard deviation of the AUC across individuals ranged from 26% (Breakout) to 84% (Asterix) of its mean. The paired design removes this variation from the comparisons.

Much of this variation was associated with the learning rate (Fig. 5). The mean AUC of DQN at a learning rate of 10^{-4} was 1.7 to 5.9 times its mean at 10^{-3} (Breakout 86.5 against 52.2, Asterix 35.1 against 6.0, Freeway 205.4 against 113.2, Space Invaders 287.9 against 90.8). As a ratio of means, this spread exceeded the treatment effect in 15 of the 16 comparisons; the exception was n-step DQN in Freeway. The lowest learning rate gave the highest mean AUC for every arm in every game except n-step DQN in Asterix and Freeway.

The n-step deficit depended strongly on the learning rate. In Freeway, its mean AUC at 10^{-4} was 16.1, against 205.4 for DQN, a ratio of 0.08, compared with ratios of 0.50 and 0.63 at the two larger rates; Asterix showed the same pattern (0.09, 0.19 and 0.70). PER, by contrast, stayed below DQN at every learning rate in Freeway and Space Invaders, with ratios between 0.41 and 0.74 and no consistent trend.

TABLE IV
SECONDARY ENDPOINTS: PAIRED COMPARISONS WITH DQN

		Last ten evaluations		Final evaluation	
Game	Arm	Δ (%)	p_t	Δ (%)	p_t
Breakout	DDQN	-0.8	0.87	-4.4	0.65
	PER	$+7.0$	0.40	$+6.2$	0.62
	n-step	-4.6	0.42	-2.5	0.79
	Avg.	$+4.8$	0.31	$+3.5$	0.69
Asterix	DDQN	$+53.9$	0.0071	$+39.0$	0.060
	PER	-14.0	0.20	-27.0	0.058
	n-step	-76.4	$\mathbf{6.3 \times 10^{-4}}$	-79.3	**0.0013**
	Avg.	$+62.4$	**0.0023**	$+49.3$	0.020
Freeway	DDQN	-3.4	0.58	$+1.0$	0.92
	PER	-32.7	0.0032	-26.1	0.029
	n-step	-50.6	$\mathbf{3.1 \times 10^{-5}}$	-49.9	$\mathbf{2.0 \times 10^{-4}}$
	Avg.	$+14.8$	0.012	$+21.9$	0.0092
Space Invaders	DDQN	$+18.2$	0.0036	$+16.1$	0.060
	PER	-46.8	$\mathbf{4.5 \times 10^{-8}}$	-49.4	$\mathbf{3.3 \times 10^{-5}}$
	n-step	-12.1	0.048	-14.7	0.11
	Avg.	$+22.6$	**0.0014**	$+17.3$	0.15

Δ and p_t as in Table III; bold: $p_t \leq 0.05/16$. DDQN: Double DQN; Avg.: Averaged DQN. These endpoints are exploratory.

D. Robustness

The nine findings survive several checks. Under the per-game threshold of our proposal ($\alpha = 0.0125$), the same nine

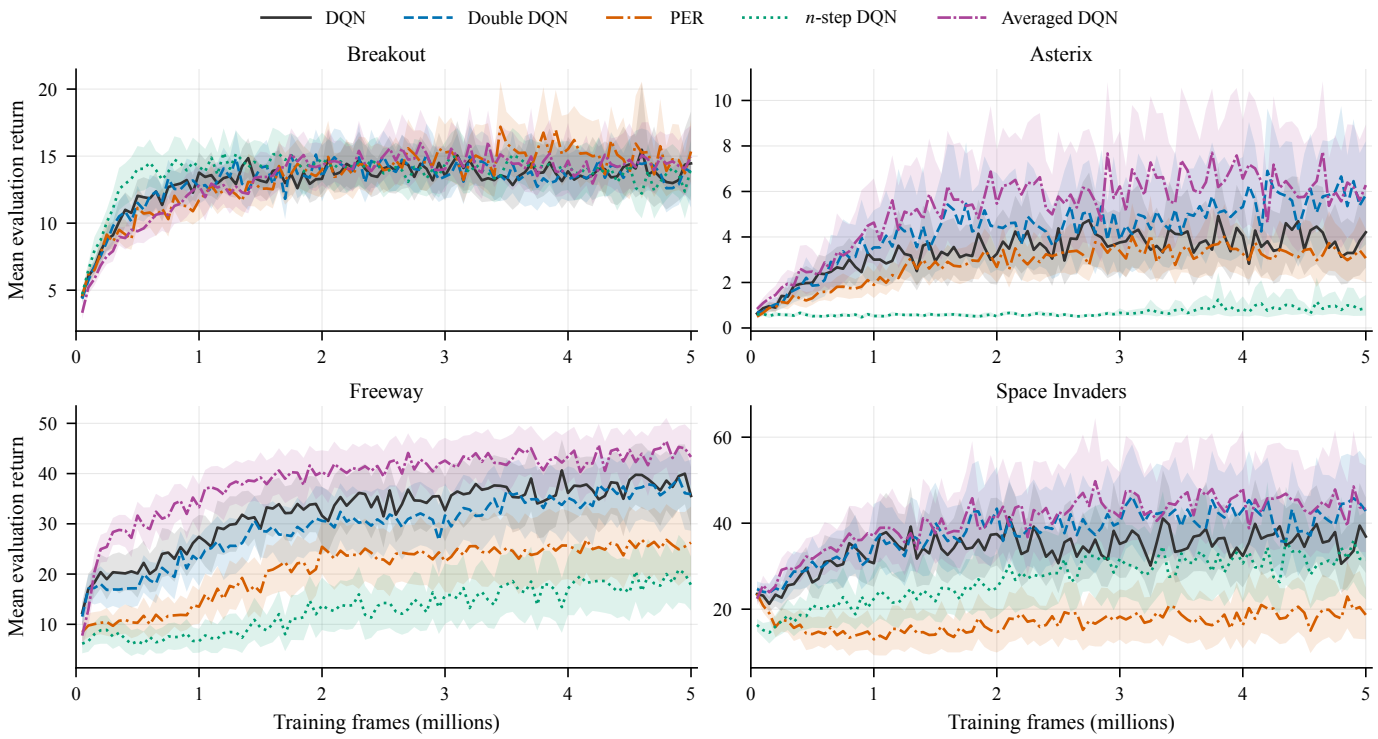


Fig. 3. Mean greedy evaluation return during training ($n = 30$ individuals per arm). Shaded bands are pointwise 95% bootstrap intervals over individuals; the curves are not smoothed.

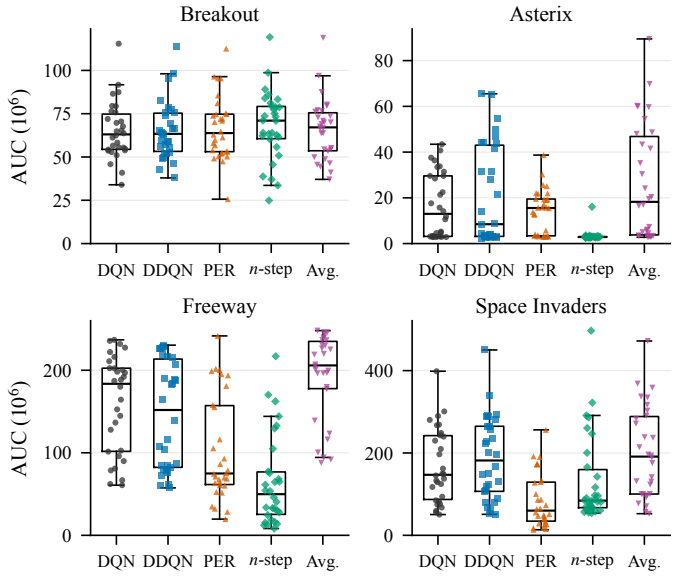


Fig. 4. AUC of every individual by arm. Boxes show medians and interquartile ranges; whiskers extend to 1.5 times the interquartile range. DDQN: Double DQN; Avg.: Averaged DQN.

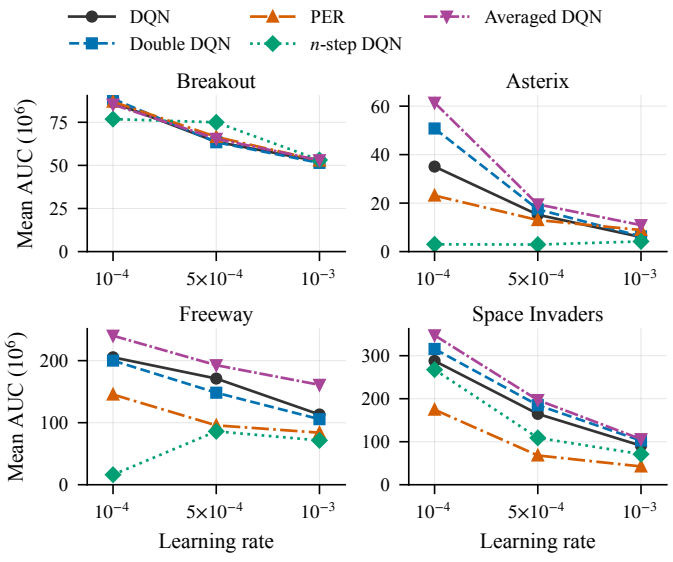


Fig. 5. Mean AUC by sampled learning rate (7, 13 and 10 individuals at 10^{-4}, 5×10^{-4} and 10^{-3}). The subgroups also differ in their other hyperparameters, so the comparison is descriptive.

comparisons are significant. The Wilcoxon test confirms eight of them at the family-wide threshold; the exception is Double DQN in Space Invaders ($p_W = 0.012$), whose interval also only just excludes zero ($[+0.1\%, +22.6\%]$), so this finding rests on the t-test alone. Restricting n-step DQN in Freeway to the individuals of the rerun (0–17) gives -53.3% ($p_t = 2.4 \times 10^{-5}$), in line with the full sample.

The secondary endpoints (Table IV) agree in direction with all nine findings, but they are noisier. With the mean of the last ten evaluations, five of the nine remain significant at the

family-wide threshold; with the final evaluation alone, only three do. No comparison that was not significant on the AUC became significant on either secondary endpoint.

E. Effect of the PER Fix

Fixing the priority bug did not change PER significantly in any game. Comparing the same 30 individuals before and after the fix, the AUC of PER changed by $+2.1\%$ in Breakout ($p_t = 0.58$), -0.9% in Asterix ($p_t = 0.89$), -9.4% in Freeway ($p_t = 0.27$) and $+4.2\%$ in Space Invaders ($p_t = 0.43$), without any correction for multiple comparisons. Before the fix, PER was already 28.6% and 52.0% below DQN in Freeway and Space Invaders, both significant at the family-wide threshold, and did not differ significantly from DQN in Breakout or Asterix. The deficit of PER is therefore not an artefact of the bug.

V. Discussion

A. RQ1: Which Extensions Help?

Averaged DQN was the only extension that improved on DQN reliably. It was better in three of the four games and on at least 24 of 30 individuals in each; the Wilcoxon test confirmed all three results, and all three kept their direction under both secondary endpoints. This is consistent with its motivation, the reduction of variance in the target [Anschel et al. 2017]. The gain has a computational cost, since each target needs ten forward passes rather than one.

Double DQN helped little. Its one significant result, in Space Invaders, was the weakest of the nine findings, and its other estimates had mixed signs. Under this recipe, decoupling action selection from evaluation was not an important source of improvement on MinAtar.

PER and n-step returns were harmful, which contrasts with their importance in the Rainbow ablation on Atari [Hessel et al. 2018] and, for n-step returns, with the MinAtar results of Obando-Ceron and Castro [2021]. Two differences between the studies may explain this, although our data cannot separate them. The first is the recipe. Schaul et al. [2016] reduced the step size for PER, and Obando-Ceron and Castro used Adam with one update every four steps. We used RMSProp with the MSE loss and one update per step, and with RMSProp they found that the Huber loss tended to work better than the MSE on Atari. Because non-uniform sampling has the same expected gradient as a different loss under uniform sampling [Fujimoto et al. 2020], PER combined with the MSE may weight large errors more heavily than intended. The second difference is the population. In Asterix and Freeway, the n-step deficit was concentrated at the lowest learning rate, which was the best learning rate for DQN, and was much smaller at 10^{-3} (Fig. 5). A study that tunes each method separately evaluates each at a single point of such a curve. We found no consistent association with the replay buffer capacity, the factor that Fedus et al. [2020] identified for n-step returns, but the subgroups are small.

In Breakout, all arms learnt quickly to a similar level. Every interval there includes zero, but their width means that small effects in either direction cannot be ruled out.

B. RQ2: What Does a Population-Level Evaluation Change?

Our results answer RQ2 in three ways. First, the endpoint matters. The final evaluation alone confirmed three of the nine AUC findings and the mean of the last ten evaluations confirmed five, although all nine kept their direction; an evaluation based on a single checkpoint would have missed most of the effects. Second, the configuration matters at least as much as the algorithm. The learning rate alone was associated with larger differences than 15 of the 16 treatment effects. Even Averaged DQN, the most consistent improvement, failed to beat DQN on 5, 6 and 4 of the 30 configurations in Asterix, Freeway and Space Invaders, so a comparison on one randomly chosen configuration would have favoured DQN with a probability of 0.13 to 0.20. Third, the paired design makes these effects detectable despite this variation: the significant effects have $|d_z|$ between 0.59 and 1.64, whereas the spread of the AUC between individuals is often as large as the differences between arms (Fig. 4).

Our proposal hypothesised that isolated effects would be smaller than those reported in the literature. This was partly supported: the benefits reported for PER and n-step returns did not appear under the shared recipe, whereas those of Averaged DQN did.

VI. Limitations

Each configuration was trained with one seed, so within an individual the effects of the configuration and the seed cannot be separated; both are shared across arms, so the comparisons remain paired. The results hold for one shared recipe with treatment settings fixed at published values; tuning each arm separately could change them. The trial covers four MinAtar games and 5×10^6 frames, so longer training or other environments could give different results, and Seaquest could not be evaluated under the present protocol. The AUC combines learning speed and final level over the observed interval. The 30 configurations are a simple random sample, so the subgroup analyses are descriptive. Finally, the DQN arm comes from the initial run; this is justified by the exact reproduction of every rerun curve, but it is a departure from running all arms together.

VII. Conclusion

In a paired randomised controlled trial of four DQN extensions over 30 sampled configurations and four MinAtar games, only Averaged DQN improved on DQN consistently. Double DQN helped only slightly, and prioritised replay and multi-step returns were harmful under the shared recipe, contrary to earlier component studies. Differences between configurations, especially in the learning rate, were larger than most treatment effects, and single-checkpoint endpoints confirmed few of the findings. Future work should add several seeds per configuration, compare recipes tuned for each arm, examine the training dynamics of PER and n-step returns directly, and evaluate Seaquest with a capped evaluation protocol.

Acknowledgement

The author thanks Steven James and Benjamin Rosman for their supervision. The experiments were run on the MSL cluster at the University of the Witwatersrand.

References

Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, pages 29304–29320, 2021.

Oron Anschel, Nir Baram, and Nahum Shimkin. Averaged-DQN: Variance reduction and stabilization for deep reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 176–185, 2017.

Cédric Colas, Olivier Sigaud, and Pierre-Yves Oudeyer. How many random seeds? Statistical power analysis in deep reinforcement learning experiments, 2018. arXiv:1806.08295.

Olive Jean Dunn. Multiple comparisons among means. *Journal of the American Statistical Association*, 56(293):52–64, 1961.

Bradley Efron. Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, 7(1):1–26, 1979.

William Fedus, Prajit Ramachandran, Rishabh Agarwal, Yoshua Bengio, Hugo Larochelle, Mark Rowland, and Will Dabney. Revisiting fundamentals of experience replay. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 3061–3071, 2020.

Scott Fujimoto, David Meger, and Doina Precup. An equivalence between loss functions and non-uniform sampling in experience replay. In *Advances in Neural Information Processing Systems*, volume 33, pages 14219–14230, 2020.

Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

J. Fernando Hernandez-Garcia and Richard S. Sutton. Understanding multi-step deep reinforcement learning: A systematic study of the DQN target, 2019. arXiv:1901.07510.

Matteo Hessel, Joseph Modayil, Hado van Hasselt, Tom Schaul, Georg Ostrovski, Will Dabney, Dan Horgan, Bilal Piot, Mohammad Azar, and David Silver. Rainbow: Combining improvements in deep reinforcement learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. arXiv:1412.6980.

Daniël Lakens. Calculating and reporting effect sizes to facilitate cumulative science: A practical primer for t-tests and ANOVAs. *Frontiers in Psychology*, 4:863, 2013.

Long-Ji Lin. Self-improving reactive agents based on reinforcement learning, planning and teaching. *Machine Learning*, 8(3–4):293–321, 1992.

Marlos C. Machado, Marc G. Bellemare, Erik Talvitie, Joel Veness, Matthew Hausknecht, and Michael Bowling. Revisiting the arcade learning environment: Evaluation protocols and open problems for general agents. *Journal of Artificial Intelligence Research*, 61:523–562, 2018.

Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

Johan S. Obando-Ceron and Pablo Samuel Castro. Revisiting Rainbow: Promoting more insightful and inclusive deep reinforcement learning research. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 1373–1383, 2021.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Arnu Pretorius, Elan van Biljon, Benjamin van Niekerk, Ryan Eloff, Matthew Reynard, Steve James, Benjamin Rosman, Herman Kamper, and Steve Kroon. If dropout limits trainable depth, does critical initialisation still matter? A large-scale statistical analysis on ReLU networks. *Pattern Recognition Letters*, 138:95–105, 2020.

Justine Rochon, Matthias Gondan, and Meinhard Kieser. To test or not to test: Preliminary assessment of normality when comparing two independent samples. *BMC Medical Research Methodology*, 12:81, 2012.

Tom Schaul, John Quan, Ioannis Antonoglou, and David Silver. Prioritized experience replay. In *International Conference on Learning Representations*, 2016. arXiv:1511.05952.

S. S. Shapiro and M. B. Wilk. An analysis of variance test for normality (complete samples). *Biometrika*, 52(3–4):591–611, 1965.

Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, USA, second edition, 2018.

Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5—RMSProp: Divide the gradient by a running average of its recent magnitude. COURSERA: Neural Networks for Machine Learning, 2012.

Mark Towers, Ariel Kwiatkowski, Jordan Terry, John U. Balis, Gianluca De Cola, Tristan Deleu, Manuel Goulão, Andreas Kallinteris, Markus Krimmel, Arjun KG, et al. Gymnasium: A standard interface for reinforcement learning environments, 2024. arXiv:2407.17032.

Hado van Hasselt, Arthur Guez, and David Silver. Deep reinforcement learning with double Q-learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 30, 2016.

Christopher J. C. H. Watkins and Peter Dayan. Technical note: Q-learning. *Machine Learning*, 8(3–4):279–292, 1992.

Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.

Kenny Young and Tian Tian. MinAtar: An Atari-inspired testbed for thorough and reproducible reinforcement learning experiments, 2019. arXiv:1903.03176.